

Algorithmique 1 — TP 2

Spécification de fonction · UE Info1.Algo1 · L1 Informatique, UPS · Semestre 1 2026-2027

Les exercices sont présentés en deux temps : **Partie 1 — Énoncés** (le code fourni tel quel dans les fichiers .py, sans les complétions demandées) puis, après un saut de page, **Partie 2 — Corrigé** (les fonctions complétées, chaque pré-/post-condition et chaque fonction ayant été exécutées réellement contre les jeux de test fournis — y compris les tests de `verifie_pre_condition` / `verifie_post_condition`, normalement commentés dans les fichiers).

Rappels — spécification de fonction

- Le **type** des paramètres et valeurs de retour.
- La **pré-condition** : condition devant être vérifiée par les paramètres afin que le problème puisse être résolu.
- La **post-condition** : condition devant être vérifiée par la (les) valeur(s) de retour.

Les assertions sont placées en tout début et en toute fin de la fonction testée :

```
def <nom de la fonction>(<param1>,<param2>):  
    assert ..., 'Pre-condition'  
    ... # CODE DE LA FONCTION  
    assert ..., 'Post-condition'  
    return ...
```

Écriture de spécification (exercices 1 à 3)

Le code de la fonction étudiée est donné ; on écrit la pré- et la post-condition dans des fonctions dédiées `verifie_pre_condition` et `verifie_post_condition` :

```
def verifie_pre_condition(...,...):  
    return ... # Pre-condition  
  
def verifie_post_condition(...,...):  
    return ... # Post-condition  
  
def <nom de la fonction>(<param1>,<param2>):  
    assert verifie_pre_condition(...,...), 'Pre-condition'  
    ... # CODE DE LA FONCTION  
    assert verifie_post_condition(...,...), 'Post-condition'  
    return ...
```

Lors d'un examen, les fonctions `test_verifie_pre_condition` et `test_verifie_post_condition` ne seront **pas fournies** (une partie des points reste possible pour une condition partiellement correcte : ex. comparaison large au lieu de stricte, encadrement incomplet, indice non vérifié avant accès à une liste). Dans ce TP, il faut **décommenter** ces fonctions de test (et leurs appels) pour vérifier ses propres conditions.

Implémentation selon une spécification (exercices 4 à 6)

La pré- et la post-condition sont données ; on complète directement le corps de la fonction. Contraintes : les paramètres ne doivent pas être modifiés, et un seul `return` est autorisé (sauf fonctions auxiliaires).

Partie 1 — Énoncés

ÉCRITURE DE SPÉCIFICATION

Exercice 1 — division euclidienne ★

Contraintes : la fonction `division_euclidienne` et les fonctions de test ne doivent pas être modifiées ; les opérateurs `//` et `%` sont interdits.

Rappel : pour a, b entiers positifs (b non nul), le quotient q et le reste r de la division euclidienne de a par b sont les uniques entiers tels que $a = q \cdot b + r$, avec $r \in [0 ; b[$.

Fichier `ex01_division_euclidienne.py` :

```
def verifie_pre_condition(a,b):
    return ...

def verifie_post_condition(a,b,q,r):
    return ...

def division_euclidienne(a,b):
    assert verifie_pre_condition(a,b), 'Pre-condition'
    q, r = 0, a
    while r >= b:
        q, r = q+1, r-b
    assert verifie_post_condition(a,b,q,r), 'Post-condition'
    return q, r
```

- 1) Compléter `verifie_pre_condition(a,b)` avec la pré-condition de `division_euclidienne`.
- 2) Compléter `verifie_post_condition(a,b,q,r)` avec la post-condition de `division_euclidienne`.
- 3) Décommenter `test_verifie_pre_condition` et `test_verifie_post_condition` (et leurs appels) pour vérifier.

Exercice 2 — calcul du minimum ★

Contraintes : la fonction `calculer_minimum` et les fonctions de test ne doivent pas être modifiées ; pour vérifier qu'un entier m appartient à `liste`, l'expression `m in liste` est interdite.

Fichier `ex02_minimum.py` :

```
def est_membre(liste,m):
    ...

def est_minorant(liste,m):
    ...

def verifie_pre_condition(liste):
    return ...

def verifie_post_condition(liste,m):
    return ...

def calculer_minimum(liste):
    assert verifie_pre_condition(liste), 'Pre-condition'
    m = liste[0]
    for e in liste:
        if e <= m:
            m = e
    assert verifie_post_condition(liste,m), 'Post-condition'
    return m
```

- 1) a) Écrire `est_membre(liste,m)` : m est-il un élément de `liste` ?
- b) Écrire `est_minorant(liste,m)` : m est-il $\leq$ tous les éléments de `liste` ?
- 2) Compléter `verifie_pre_condition(liste)` avec la pré-condition de `calculer_minimum`.
- 3) Compléter `verifie_post_condition(liste,m)` avec la post-condition de `calculer_minimum` (s'appuyer sur `est_membre` et `est_minorant`).
- 4) Décommenter `test_verifie_pre_condition` et `test_verifie_post_condition` pour vérifier.

Exercice 3 — indice de stricte croissance ★

Contrainte : la fonction `indice_stricte_croissance` et les fonctions de test ne doivent pas être modifiées. Fichier `ex03_indice_stricte_croissance.py` : accepte une liste non vide dont le **dernier élément est strictement supérieur au premier**, et retourne un indice `i` tel que `liste[i]` est strictement supérieur à son prédécesseur.

```
def verifie_pre_condition(liste):
    return ...

def verifie_post_condition(liste,i):
    return ...

def indice_stricte_croissance(liste):
    assert verifie_pre_condition(liste), 'Pre-condition'
    i = 1
    while liste[i-1]>=liste[i]:
        i += 1
    assert verifie_post_condition(liste,i), 'Post-condition'
    return i
```

- 1) Compléter `verifie_pre_condition(liste)`. Indication : vérifier la validité de la taille de la liste avant d'en comparer les éléments.
- 2) Compléter `verifie_post_condition(liste,i)`. Indications : vérifier la validité de l'indice `i` avant de comparer des éléments ; il peut exister plusieurs indices valides.
- 3) Décommenter `test_verifie_pre_condition` et `test_verifie_post_condition` pour vérifier.

Remarque : la pré-condition choisie ici est une condition *suffisante* mais pas *nécessaire* : pour `[8,3,5,1]`, la pré-condition n'est pas respectée, et pourtant `i=2` convient (`5 > 3`). Ce n'est donc pas la plus faible pré-condition possible.

IMPLÉMENTATION SELON UNE SPÉCIFICATION

Exercice 4 — réciproques entières ★

On cherche la partie entière d'une racine carrée et d'un logarithme, sans utiliser aucun calcul flottant (toutes les variables doivent être de type `int`) — par une boucle balayant de façon croissante les solutions possibles.

Fichier `ex04_reciproques_entieres.py` :

```
def racine_entiere(n):
    assert n>=0, 'Pre-condition'
    ...
    assert a>=0 and a**2<=n<(a+1)**2, 'Post-condition'
    return a

def log2_entier(n):
    assert n>0, 'Pre-condition'
    ...
    assert k>=0 and 2**k<=n<2**(k+1), 'Post-condition'
    return k

def logb_entier(n,b):
    assert n>0 and b>=2, 'Pre-condition'
    ...
    assert k>=0 and b**k<=n<b**(k+1), 'Post-condition'
    return k
```

- 1) Compléter `racine_entiere(n)` (racine carrée entière).
- 2) Compléter `log2_entier(n)` (logarithme entier en base 2).
- 3) Compléter `logb_entier(n,b)` (logarithme entier en base `b` quelconque, $b \geq 2$).

(La fonction de test se contente d'appeler la fonction, sans vérifier elle-même le résultat : c'est l'assertion de post-condition qui fait ce travail.)

Exercice 5 — médiane ★

On appelle **médiane** d'une liste d'entiers toute valeur de cette liste telle qu'au moins la moitié des éléments lui sont inférieurs ou égaux, et qu'au moins la moitié lui sont supérieurs ou égaux. (Définition volontairement différente de la médiane mathématique habituelle : on évite les flottants et on choisit une valeur présente dans la liste.)

- La médiane de [5,4,6,2,0] est 4 (3 valeurs ≤ 4 : 4,2,0 ; et 3 valeurs ≥ 4 : 5,4,6).
- Les deux médianes acceptées pour [5,2,7,4] sont 4 et 5.
- La médiane de [1,3,3,5,2,3] est 3 (5 valeurs ≤ 3 et 4 valeurs ≥ 3).

Fichier ex05_mediane.py :

```
def nombre_inferieurs(liste,valeur):
    ...

def nombre_superieurs(liste,valeur):
    ...

def calculer_mediane(liste):
    n = len(liste)
    assert n>0, 'Pre-condition'
    ...
    assert (
        nombre_inferieurs(liste,mediane)>=(n+1)//2
        and nombre_superieurs(liste,mediane)>=(n+1)//2
    ), 'Post-condition'
    return mediane
```

- 1) Compléter les fonctions auxiliaires nombre_inferieurs(liste,valeur) (nombre d'éléments $\leq$ valeur) et nombre_superieurs(liste,valeur) (nombre d'éléments $\geq$ valeur).
- 2) Compléter calculer_mediane(liste) (liste non vide, de longueur n). On s'autorise ici à utiliser nombre_inferieurs et nombre_superieurs dans le corps de la fonction, pas seulement dans les assertions.

Contrainte : ne pas modifier les variables liste et n.

Exercice 6 — indice du minimum ★

On veut déterminer l'indice où se trouve le minimum d'une liste.

Fichier ex06_indice_minimum.py :

```
def est_minorant_plage(liste,d,f,m):
    ...

def indice_minimum(liste):
    n = len(liste)
    assert n>0, 'Pre-condition'
    ...
    assert 0<=i_min<n and est_minorant_plage(liste,0,n,liste[i_min]), 'Post-condition'
    return i_min
```

- 1) Compléter est_minorant_plage(liste,d,f,m) : True si m est $\leq$ tous les éléments d'indice i tel que $d \leq i < f$ (plage [d;f[, d inclus, f exclu), False sinon.
- 2) a) Compléter indice_minimum(liste) (liste non vide, longueur n) qui retourne n'importe quel indice i_min du minimum (pas nécessairement unique).
- b) Compléter indice_premier_minimum(liste), dont la post-condition ajoute que tous les éléments d'indices inférieurs à i_min sont strictement supérieurs au minimum : est_minorant_plage(liste,0,i_min,liste[i_min]+1).
- c) Compléter indice_dernier_minimum(liste) (cas symétrique, avec est_minorant_plage(liste,i_min+1,n,liste[i_min]+1)).

Contraintes (pour les 3 sous-questions) : est_minorant_plage ne doit être appelée nulle part ailleurs que dans la post-condition ; les variables liste et n ne doivent pas être modifiées.

EXERCICES D'ENTRAÎNEMENT

Cette partie mélange écriture de spécification et implémentation selon une spécification.

Exercice 7 — indice d'encadrement ★

Contrainte : la fonction `indice_encadrement` et les fonctions de test ne doivent pas être modifiées.
Fichier `ex07_indice_encadrement.py` : accepte une liste non vide et un entier valeur supérieur ou égal au premier élément de liste et strictement inférieur au dernier. Retourne un indice `i` tel que `valeur ≥ liste[i-1]` et `valeur < liste[i]`.

```
def verifie_pre_condition(liste,valeur):
    return ...

def verifie_post_condition(liste,valeur,i):
    return ...

def indice_encadrement(liste,valeur):
    assert verifie_pre_condition(liste,valeur), 'Pre-condition'
    i = 1
    while liste[i-1]>valeur or valeur>=liste[i]:
        i += 1
    assert verifie_post_condition(liste,valeur,i), 'Post-condition'
    return i
```

- 1) Compléter `verifie_pre_condition(liste,valeur)`. Indication : vérifier la validité de la taille de la liste avant d'en comparer les éléments.
- 2) Compléter `verifie_post_condition(liste,valeur,i)`. Indications : vérifier la validité de l'indice `i` avant de comparer des éléments ; il peut exister plusieurs indices valides.
- 3) Décommenter `test_verifie_pre_condition` et `test_verifie_post_condition` pour vérifier.

Exercice 8 — compression RLE ★★ (avancé)

La fonction `decompresser_liste` est vérifiée par des tests unitaires ; `compresser_liste` l'est par un test de post-condition s'appuyant sur `decompresser_liste`.

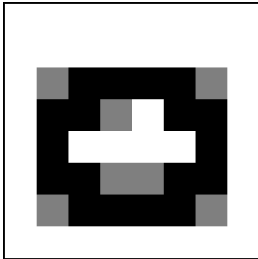


Image carrée en niveaux de gris de 8 pixels de côté (0 = noir, 255 = blanc, valeurs intermédiaires = gris) :

```
255 255 255 255 255 255 255 255
255 255 255 255 255 255 255 255
255 127 0 0 0 0 127 255
255 0 0 127 255 0 0 255
255 0 255 255 255 255 0 255
255 0 0 127 127 0 0 255
255 127 0 0 0 0 127 255
255 255 255 255 255 255 255 255
```

La méthode **RLE** (Run Length Encoding) encode une plage de nombres identiques par le couple [longueur de la plage, nombre répété]. Exemple : [255,255,0,0,0,0,63,63,63,63,127,127,127] s'encode [2,255,4,0,4,63,3,127].

Fichier `ex08_rle.py` :

```
def decompresser_liste(liste_compressee):
    ...

def compresser_liste(liste):
    assert True, 'Pre-condition'
    ...
    assert decompresser_liste(liste_compressee)==liste, 'Post-condition'
    return liste_compressee
```

- 1) Compléter `decompresser_liste(liste_compressee)` : ne modifie pas la liste passée en paramètre. Une liste compressée est valide si elle est de longueur paire, si les éléments d'indices pairs (longueurs) sont strictement positifs, et si deux éléments consécutifs d'indices impairs (valeurs) sont distincts. Si la liste n'est

pas valide, retourner None.

- 2) Écrire `compresser_liste(liste)` : pas de pré-condition (toute liste doit pouvoir être traitée) ; post-condition `decompresser_liste(liste_compressée)==liste` ; ne modifie pas liste.
- 3) Taux de compression $T = 1 - (\text{taille compressée} / \text{taille originale})$. a) Calculer T sur l'image 8×8 ci-dessus. b) Afficher le résultat sous forme de pourcentage.

Exercice 9 — principe des tiroirs ★★ (avancé)

Le **principe des tiroirs** de Dirichlet affirme que si n chaussettes sont rangées dans m tiroirs avec $m < n$, alors un tiroir contient au moins deux chaussettes.

- `trouver_elements_identiques(liste)` : liste de longueur n , valeurs parmi $n-1$ possibilités (de 0 à $n-2$) ; trouve deux indices distincts partageant la même valeur.
- `trouver_valeur_absente(liste)` : liste de longueur n , valeurs parmi $n+1$ possibilités (de 0 à n) ; trouve l'entier entre 0 et n absent de la liste.

Fichier `ex09_tiroirs.py` :

```
def est_liste_bornee(liste,m1,m2):
    ...

def est_valeur_presente(liste,valeur):
    ...

def trouver_elements_identiques(liste):
    n = len(liste)
    assert n>0 and est_liste_bornee(liste,0,n-2), 'Pre-condition'
    ...
    assert 0<=i1<i2<n and liste[i1]==liste[i2]
    return i1,i2

def trouver_valeur_absente(liste):
    n = len(liste)
    assert est_liste_bornee(liste,0,n), 'Pre-condition'
    ...
    assert 0<=valeur<=n and not est_valeur_presente(liste,valeur)
    return valeur
```

- 1) Écrire `est_liste_bornee(liste,m1,m2)` (True si tous les éléments e vérifient $m1 \leq e \leq m2$) et `est_valeur_presente(liste,valeur)` (True si valeur figure dans liste).
- 2) Compléter `trouver_elements_identiques(liste)` : trouver $i1,i2$ avec $0 \leq i1 < i2 < n$ et `liste[i1]==liste[i2]`.
Exemple : `liste=[2,0,3,4,5,3,1]` ($n=7$) $\rightarrow i1=2, i2=5$ (valeur 3).
- 3) Compléter `trouver_valeur_absente(liste)` : trouver valeur avec $0 \leq \text{valeur} \leq n$ et valeur absente de liste.
Exemple : `liste=[2,0,3,7,5,6,1]` ($n=7$) $\rightarrow \text{valeur}=4$.

Exercice 10 — les quatre carrés de Lagrange ★★★ (expert)

Le théorème des quatre carrés de Lagrange (conjecture de Bachet) affirme que tout entier positif ou nul n peut s'écrire comme somme de quatre carrés : $n = a^2 + b^2 + c^2 + d^2$.

Fichier `ex10_lagrange.py` :

```
def quatre_carres(n):
    assert n>=0, 'Pre-condition'
    ...
    assert a**2+b**2+c**2+d**2==n, 'Post-condition'
    return a,b,c,d
```

Compléter `quatre_carres(n)` qui accepte un entier n et retourne quatre entiers a, b, c, d vérifiant la post-condition ci-dessus.

Partie 2 — Corrigé

ÉCRITURE DE SPÉCIFICATION

Exercice 1 — division euclidienne ★

Pré-condition : a et b doivent être positifs, et b non nul :

```
def verifie_pre_condition(a,b):  
    return a>=0 and b>=1
```

Post-condition : le reste doit être dans $[0 ; b[$, et l'égalité $a = q \cdot b + r$ doit être vérifiée :

```
def verifie_post_condition(a,b,q,r):  
    return 0<=r<b and a==q*b+r
```

Vérifié en exécutant test_verifie_pre_condition, test_verifie_post_condition et test_division_euclidienne (décommentés) : les 3 passent sans erreur.

Exercice 2 — calcul du minimum ★

1) Fonctions auxiliaires (boucle explicite, sans l'opérateur in) :

```
def est_membre(liste,m):  
    for e in liste:  
        if e == m:  
            return True  
    return False  
  
def est_minorant(liste,m):  
    for e in liste:  
        if e < m:  
            return False  
    return True
```

2) Pré-condition : la liste ne doit pas être vide :

```
def verifie_pre_condition(liste):  
    return len(liste)>0
```

3) Post-condition : m doit être à la fois un élément de la liste et un minorant de la liste (les deux ensemble caractérisent exactement le minimum) :

```
def verifie_post_condition(liste,m):  
    return est_membre(liste,m) and est_minorant(liste,m)
```

Vérifié en exécutant les 5 fonctions de test (est_membre, est_minorant, verifie_pre_condition, verifie_post_condition, calculer_minimum) : toutes passent.

Exercice 3 — indice de stricte croissance ★

1) Pré-condition : taille ≥ 2 (vérifiée en premier, court-circuit du and) puis dernier élément strictement supérieur au premier :

```
def verifie_pre_condition(liste):  
    return len(liste)>=2 and liste[-1]>liste[0]
```

2) Post-condition : l'indice i doit d'abord être valide ($1 \leq i < \text{longueur}$, vérifié en premier), puis on compare liste[i-1] et liste[i] :

```
def verifie_post_condition(liste,i):  
    return 1<=i<len(liste) and liste[i-1]<liste[i]
```

Vérifié sur les 26 assertions fournies (8 pour la pré-condition, 18 pour la post-condition) ainsi que sur test_indice_stricte_croissance : tout passe.

IMPLÉMENTATION SELON UNE SPÉCIFICATION

Exercice 4 — réciproques entières ★

Même idée dans les 3 cas : balayer les solutions possibles par une boucle croissante, en s'arrêtant juste avant de dépasser n.

```
def racine_entiere(n):
    assert n>=0, 'Pre-condition'
    a = 0
    while (a+1)**2 <= n:
        a += 1
    assert a>=0 and a**2<=n<(a+1)**2, 'Post-condition'
    return a

def log2_entier(n):
    assert n>0, 'Pre-condition'
    k = 0
    while 2**(k+1) <= n:
        k += 1
    assert k>=0 and 2**k<=n<2**(k+1), 'Post-condition'
    return k

def logb_entier(n,b):
    assert n>0 and b>=2, 'Pre-condition'
    k = 0
    while b**(k+1) <= n:
        k += 1
    assert k>=0 and b**k<=n<b**(k+1), 'Post-condition'
    return k
```

Vérifié par exécution : racine_entiere sur $n=0..999$, log2_entier sur $n=1..999$ et $\log_2 \text{entier}(2^{100}-1)=99$, logb_entier sur $n=1..99 \times b=2..10$ et deux cas extrêmes ($\log_b \text{entier}(2^{100}-1,2)=99$, $\log_b \text{entier}(5^{100}-1,5)=99$) : toutes les assertions passent.

Exercice 5 — médiane ★

1) Fonctions auxiliaires (comptage par balayage) :

```
def nombre_inferieurs(liste,valeur):
    compte = 0
    for e in liste:
        if e<=valeur:
            compte += 1
    return compte

def nombre_supérieurs(liste,valeur):
    compte = 0
    for e in liste:
        if e>=valeur:
            compte += 1
    return compte
```

2) On trie une copie de la liste et on prend l'élément central (index $(n-1)//2$, arrondi vers le bas) : cette valeur est toujours un élément de la liste, et par construction du tri au moins la moitié des éléments lui sont $\leq$ et au moins la moitié lui sont $\geq$.

```
def calculer_mediane(liste):
    n = len(liste)
    assert n>0, 'Pre-condition'
    trie = sorted(liste)
    mediane = trie[(n-1)//2]
    assert (
        nombre_inferieurs(liste,mediane)>=(n+1)//2
        and nombre_supérieurs(liste,mediane)>=(n+1)//2
    ), 'Post-condition'
    return mediane
```

sorted(liste) crée une nouvelle liste sans toucher à liste ni à n (contrainte respectée). Vérifié sur les 8 cas unitaires fournis, puis sur 200 listes aléatoires (100 de longueur 24, 100 de longueur 25) vérifiées via la post-condition : tout passe.

Exercice 6 — indice du minimum ★

1) `est_minorant_plage` : on ne parcourt que la plage `[d ; f[` :

```
def est_minorant_plage(liste,d,f,m):
    for i in range(d,f):
        if liste[i]<m:
            return False
    return True
```

2) a) **Balayage classique, en gardant le premier indice qui bat le minimum courant (comparaison stricte `<`)** : cela produit naturellement le *premier* indice du minimum.

```
def indice_minimum(liste):
    n = len(liste)
    assert n>0, 'Pre-condition'
    i_min = 0
    for i in range(1,n):
        if liste[i] < liste[i_min]:
            i_min = i
    assert 0<=i_min<n and est_minorant_plage(liste,0,n,liste[i_min]), 'Post-condition'
    return i_min
```

b) `indice_premier_minimum` utilise exactement le même algorithme que `indice_minimum` : la comparaison stricte `<` garantit déjà qu'on s'arrête sur le *premier* indice atteignant le minimum.

```
def indice_premier_minimum(liste):
    n = len(liste)
    assert n>0, 'Pre-condition'
    i_min = 0
    for i in range(1,n):
        if liste[i] < liste[i_min]:
            i_min = i
    assert (
        0<=i_min<n and est_minorant_plage(liste,0,n,liste[i_min])
        and est_minorant_plage(liste,0,i_min,liste[i_min]+1)
    ), 'Post-condition'
    return i_min
```

c) `indice_dernier_minimum` : même algorithme, mais avec une comparaison large `≤` pour que chaque nouvelle occurrence du minimum (même égale) écrase la précédente — on obtient ainsi la *dernière* occurrence.

```
def indice_dernier_minimum(liste):
    n = len(liste)
    assert n>0, 'Pre-condition'
    i_min = 0
    for i in range(1,n):
        if liste[i] <= liste[i_min]:
            i_min = i
    assert (
        0<=i_min<n and est_minorant_plage(liste,0,n,liste[i_min])
        and est_minorant_plage(liste,i_min+1,n,liste[i_min]+1)
    ), 'Post-condition'
    return i_min
```

Aucune des 3 fonctions n'appelle `est_minorant_plage` dans son corps (seulement dans l'assertion de post-condition, déjà fournie) ; `liste` et `n` ne sont jamais modifiées. Vérifié sur tous les cas unitaires fournis (y compris la liste de 246913 éléments) et sur 300 listes aléatoires supplémentaires : tout passe.

EXERCICES D'ENTRAÎNEMENT

Exercice 7 — indice d'encadrement ★

Pré-condition : $\text{taille} \geq 2$ (court-circuit), puis valeur encadrée au sens large-strict par le premier et le dernier élément :

```
def verifie_pre_condition(liste,valeur):
    return len(liste)>=2 and liste[0]<=valeur<liste[-1]
```

Post-condition : indice valide en premier, puis valeur encadrée entre $\text{liste}[i-1]$ (inclus) et $\text{liste}[i]$ (exclu) :

```
def verifie_post_condition(liste,valeur,i):
    return 1<=i<len(liste) and liste[i-1]<=valeur<liste[i]
```

Vérifié sur les 6 assertions de pré-condition, les 14 de post-condition et test_indice_encadrement : tout passe.

Exercice 8 — compression RLE ★★ (avancé)

1) décompresser_liste : on vérifie la validité (longueur paire, longueurs strictement positives, valeurs consécutives distinctes) avant de reconstruire la liste décompressée :

```
def decompresser_liste(liste_compressee):
    if len(liste_compressee) % 2 != 0:
        return None
    valeurs_repetees = []
    for i in range(0, len(liste_compressee), 2):
        longueur = liste_compressee[i]
        if longueur <= 0:
            return None
        valeurs_repetees.append(liste_compressee[i+1])
    for i in range(1, len(valeurs_repetees)):
        if valeurs_repetees[i] == valeurs_repetees[i-1]:
            return None
    resultat = []
    for i in range(0, len(liste_compressee), 2):
        longueur = liste_compressee[i]
        valeur = liste_compressee[i+1]
        resultat += [valeur]*longueur
    return resultat
```

2) compresser_liste : on balaie la liste en regroupant chaque plage de valeurs identiques consécutives (deux indices i, j qui délimitent la plage courante) :

```
def compresser_liste(liste):
    assert True, 'Pre-condition'
    liste_compressee = []
    i = 0
    n = len(liste)
    while i < n:
        j = i
        while j < n and liste[j] == liste[i]:
            j += 1
        liste_compressee += [j-i, liste[i]]
        i = j
    assert decompresser_liste(liste_compressee)==liste, 'Post-condition'
    return liste_compressee
```

Vérifié sur les 7 cas unitaires de decompresser_liste (dont les 3 cas de liste invalide → None), les 4 cas unitaires de compresser_liste, et 100 listes aléatoires (jusqu'à 100 plages de longueur 1 à 100) vérifiées par la post-condition. Ni liste ni liste_compressee ne sont modifiées en dehors de leur propre construction.

3) Taux de compression sur l'image 8×8 :

```
image_compressee = compresser_liste(image)
taux = 1 - len(image_compressee)/len(image)
print("{:.1%}".format(taux))
```

Résultat obtenu par exécution : l'image compressée est [17, 255, 1, 127, 4, 0, 1, 127, 2, 255, 2, 0, 1, 127, 1, 255, 2, 0, 2, 255, 1, 0, 4, 255, 1, 0, 2, 255, 2, 0, 2, 127, 2, 0, 2, 255, 1, 127, 4, 0, 1, 127, 9, 255] (44 entiers), contre 64 dans l'image originale, soit un **taux de compression T** $\approx 31,2\%$ ($1 - 44/64$).

Exercice 9 — principe des tiroirs ★★ (avancé)

1) Fonctions auxiliaires :

```
def est_liste_bornee(liste, m1, m2):
    for e in liste:
        if not (m1 <= e <= m2):
            return False
    return True

def est_valeur_presente(liste, valeur):
    for e in liste:
        if e == valeur:
            return True
    return False
```

2) trouver_elements_identiques : les valeurs sont dans $[0 ; n-2]$, donc $n-1$ valeurs possibles pour n éléments — on mémorise, pour chaque valeur v , le premier indice où elle apparaît (tableau vu indexé par v , principe des tiroirs : les tiroirs sont les valeurs $0..n-2$, les chaussettes les indices $0..n-1$).

```
def trouver_elements_identiques(liste):
    n = len(liste)
    assert n > 0 and est_liste_bornee(liste, 0, n-2), 'Pre-condition'
    vu = [-1] * (n-1)
    i1, i2 = 0, 0
    for i in range(n):
        v = liste[i]
        if vu[v] != -1:
            i1, i2 = vu[v], i
            break
        vu[v] = i
    assert 0 <= i1 < i2 < n and liste[i1] == liste[i2]
    return i1, i2
```

3) trouver_valeur_absente : même idée avec $n+1$ valeurs possibles ($0..n$) pour n éléments — on marque les valeurs présentes puis on cherche la première valeur non marquée.

```
def trouver_valeur_absente(liste):
    n = len(liste)
    assert est_liste_bornee(liste, 0, n), 'Pre-condition'
    vu = [False] * (n+1)
    for e in liste:
        vu[e] = True
    valeur = 0
    while vu[valeur]:
        valeur += 1
    assert 0 <= valeur <= n and not est_valeur_presente(liste, valeur)
    return valeur
```

Vérifié sur tous les cas unitaires fournis (y compris l'exemple $[2,0,3,4,5,3,1] \rightarrow (2,5)$ et $[2,0,3,7,5,6,1] \rightarrow 4$) et sur 200 listes aléatoires supplémentaires : tout passe.

Exercice 10 — les quatre carrés de Lagrange ★★★ (expert)

Recherche exhaustive bornée par `math.isqrt` : on essaie a du plus grand possible vers 0, puis pour chaque a on essaie b , puis c , et on déduit d en testant si le reste est un carré parfait. Dès qu'une solution est trouvée, on sort des 3 boucles via un indicateur testé *après* chaque boucle interne (et non au début de la boucle suivante, ce qui écraserait la variable trouvée par la valeur de l'itération suivante).

```

import math

def quatre_carres(n):
    assert n>=0, 'Pre-condition'
    a = b = c = d = 0
    amax = math.isqrt(n)
    for a in range(amax, -1, -1):
        reste_a = n - a*a
        bmax = math.isqrt(reste_a)
        trouve = False
        for b in range(bmax, -1, -1):
            reste_b = reste_a - b*b
            cmax = math.isqrt(reste_b)
            for c in range(cmax, -1, -1):
                reste_c = reste_b - c*c
                d = math.isqrt(reste_c)
                if d*d == reste_c:
                    trouve = True
                    break
            if trouve:
                break
        if trouve:
            break
    assert a**2+b**2+c**2+d**2==n, 'Post-condition'
    return a,b,c,d

```

Piège rencontré et corrigé pendant la vérification : une première version testait `if trouve: break` en tout début de boucle (comme condition d'entrée de l'itération suivante). Or en Python, la variable de boucle est déjà réaffectée à la valeur suivante *avant* l'exécution du corps ; le test échouait alors pour $n=1$ (a valait 0 au lieu de 1 au moment du retour). Déplacer le test juste après chaque boucle interne (tant que la même valeur de a/b est encore active) corrige le problème. Vérifié pour tout n de 0 à 999 : la post-condition est vraie à chaque fois (ex. `quatre_carres(0)=(0,0,0,0)`, `quatre_carres(1)=(1,0,0,0)`, `quatre_carres(7)=(2,1,1,1)`, `quatre_carres(999)=(31,6,1,1)`), en moins de 10 ms au total.

À retenir

- **Spécifier avant d'implémenter** : une pré-condition délimite le domaine où le problème a un sens, une post-condition caractérise le résultat attendu — les deux se vérifient indépendamment de l'algorithme choisi pour les satisfaire.
- **Valider l'ordre des tests** : toujours vérifier la taille/validité d'un indice *avant* de comparer des éléments — l'évaluation paresseuse du `and/or` en Python évite alors une `IndexError` (exercices 3 et 7).
- **Fonctions auxiliaires** : factoriser une propriété réutilisable (`est_membre`, `est_minorant`, `est_minorant_plage`, `est_liste_bornee`...) rend la post-condition lisible et souvent réutilisable dans le corps même de la fonction (exercice 5).
- **Une seule pré-/post-condition peut avoir plusieurs implémentations valides** : `indice_minimum`, `indice_premier_minimum` et `indice_dernier_minimum` ne diffèrent que par une comparaison stricte ou large dans la boucle de recherche.
- **Piège classique en Python** : tester une condition de sortie *en tête* d'une boucle `for` ne préserve pas la valeur de la variable de boucle de l'itération précédente — toujours sortir juste après avoir trouvé le résultat, pas au début du tour suivant (exercice 10).
- **Le principe des tiroirs** se traduit en code par un tableau « vu » indexé directement par les valeurs possibles : c'est souvent plus simple et plus rapide qu'une recherche par paires imbriquées (exercice 9).